

Sentiment analysis of Amazon reviews using PyTorch

A comparison of traditional and deep learning models

Sander Oppen and Ole Markus Bråthen
University of Oslo

(Dated: January 19, 2026)

Sentiment analysis is a well-studied problem in the field of natural language processing (NLP), with a wide variety of real world applications. Due to the complexity and nuances of language, it can often be difficult to get well-performing models using classic NLP methods. We look at how three different models: Bag of Words (BoW), Recurrent Neural Networks (RNN), and Long Short-Term Memory (LSTM) models perform on the task of classifying Amazon product reviews by their star rating using the AmazonReviewFull dataset. We train on a set of 3 000 000 reviews across all product ranges equally distributed among the five rating classes. We produce a LSTM model with 61,3% accuracy over the test set, a slight improvement on previous research. We compare with more traditional BoW and plain RNN models, none of which achieve near the same accuracy.

I. INTRODUCTION

Natural Language Processing has seen rapid advancements in recent years, becoming an important area of application for machine learning. The task of sentiment analysis, determining the emotional tone behind a body of text, is a challenge with many real-world applications. While binary sentiment classification (positive/negative) is common, multiclass problems like predicting exact star ratings are more challenging due to the finer-grained distinctions required.

In this project, we address the problem of multiclass sentiment classification, specifically aiming to predict the star rating (1 to 5) of Amazon product reviews based solely on their textual content. To tackle this sequence modeling problem, we implement and compare three distinct architectures using the PyTorch framework. We begin with a traditional Bag of Words model to serve as a point of comparison. We then progress to deep learning approaches designed for sequential data: the standard Recurrent Neural Network (RNN) and the Long Short-Term Memory (LSTM) network. The latter is specifically chosen to address the known limitations of simple RNNs, such as the vanishing gradient problem, which often hinders performance on longer texts.

We utilize the Amazon Review Full dataset [1], a large-scale collection containing millions of reviews. Our analysis focuses on the trade-offs between model complexity and generalization, exploring how well these different architectures capture the semantic dependencies required to distinguish between similar star ratings. Comparisons to previous experiments on the same dataset are also done, and we find that our LSTM model is a slight improvement on previously presented models on this dataset. All model implementations, code used for producing plots, and other utilities can be found in the project repository.

II. METHODS

A. Data

1. Dataset

We have used the Amazon reviews dataset produced by Zhang et al. [1], which is a subset of a larger dataset from the Stanford Network Analysis Project (SNAP)[2]. For each of the star ratings (1-5), there are 600 000 testing samples in each class, as well as 130 000 testing samples for each class. All reviews contain both the review title and the review text as input. The dataset was accessed through the Torchtext library, allowing for easy integration with the rest of the PyTorch models. [3]

2. Preprocessing

Data is tokenized into words, where we use a standardized alphabet for all text. The implementation of this is taken from Raschka et al. [4]. The alphabet is constrained in the tokenizer, removing many special characters, significantly simplifying the text formatting. The characters we allow, and thereby our alphabet is given by the following characters:

```
abcdefghijklmnopqrstuvwxyz  
0123456789_ : ; = ( )
```

Almost all punctuation is removed, however some special characters are still allowed, due to their use as emoticons, like :), :(, and :D. These can provide a strong indicator of the review sentiment, and the added complexity of the larger alphabet is therefore a reasonable tradeoff. HTML markup is also removed, ensuring that we are only processing the human-written text for each review. Text is tokenized based on words, where we consider any space-separated string of characters to be one whole word. After tokenization, all word frequencies are calculated, and words are assigned a numerical

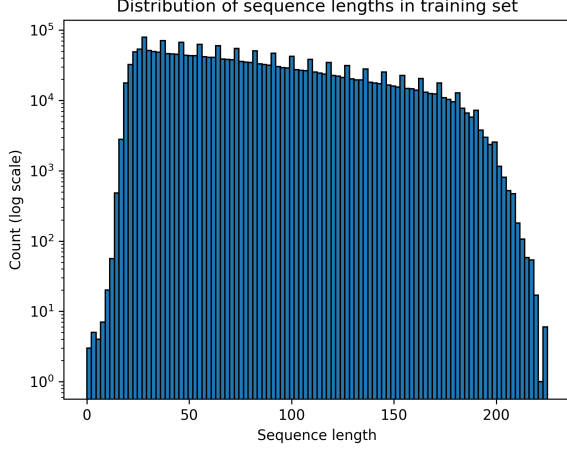


Figure 1. Counts of sequence length in the training dataset after preprocessing, with outliers removed.

value based on their order in the frequency list. Unknown characters are also given a special label, as we might encounter words in the test data that are not present in the training data.

For the dataset, sequences are distributed relatively equally over a reasonable distribution of different lengths, as illustrated in Figure 1. This makes the dataset well suited for evaluating networks for sentiment analysis, as there is a large variety of text styles present. A good model must both be able to infer sentiment from few words, as well as from large texts.

B. Activation functions

1. Linear

For simple fully connected layers in the neural networks, we use a linear activation function given by

$$f(x) = x. \quad (1)$$

This function does no normalization of the data, and the value of a neuron is therefore determined linearly by the weights of the previous layer. This function is used as the final layer activation function for all models, due to us using cross entropy for our loss function, which expects an unnormalized vector as input.

2. ReLU

For larger models with multiple layers, we use the Rectified Linear Unit as an activation function between the hidden layers. This is given by

$$\text{ReLU}(z) = \begin{cases} z & z > 0 \\ 0 & z \leq 0 \end{cases} \quad (2)$$

This activation function is known for helping prevent the problem of vanishing gradients which RNNs are known to suffer from, by only keeping positive values. This function is also simple computationally, speeding up the training process.

3. Sigmoid

The sigmoid activation function is given by

$$\sigma(z) = \frac{1}{1 + e^{-z}}. \quad (3)$$

This function takes an input value and transforms it into a value in the range $(-1, 1)$. We will not use this explicitly, however it is used in the recurrence layer of LSTM networks.

4. Hyperbolic tangent

The tanh (hyperbolic tangent) function is similar to the sigmoid function, in that it takes input values and transform them into the range $(-1, 1)$. It is given by

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}. \quad (4)$$

This will also only be used implicitly in the recurrence of LSTM networks.

C. Optimizers

1. Stochastic Gradient Descent

Stochastic Gradient Descent (SGD) is a simple optimization algorithm used for minimizing the loss function in neural networks. Unlike standard gradient descent, which computes the gradient using the entire dataset, SGD approximates the gradient using a small batch of data, making it computationally efficient for large datasets.

The algorithm takes as input the objective function or weights $f_t(\theta)$ to minimize, the current parameters θ_t , and the learning rate η . The algorithm then calculates

$$t = t + 1 \quad (5)$$

$$\mathbf{g}_t = \nabla_{\theta} f_t(\theta_{t-1}) \quad (6)$$

$$\theta_t = \theta_{t-1} - \eta \mathbf{g}_t \quad (7)$$

Here $\mathbf{g}_t$ represents the gradient of the objective function with respect to the parameters at the current step. As the learning rate η is fixed, finding the optimal value for this can greatly affect the model performance.

2. Adam

ADAM is a more complex optimizer, best suited for problems with noisy gradients.[5] It uses two momentum variables $\mathbf{s} = 0$ and $\mathbf{r} = 0$ for decaying gradients and squared gradients. In addition, some bias correction is computed each iteration, based on the iteration number t . The algorithm takes as input the objective function or weights $f_t(\theta)$ to minimize, the current parameters θ_t , the learning rate η , and the hyperparameters β_1, β_2 . The algorithm then calculates

$$t = t + 1 \quad (8)$$

$$\mathbf{g}_t = \nabla_{\theta} f_t(\theta_{t-1}) \quad (9)$$

$$\mathbf{s} = \beta_1 \mathbf{s} + (1 - \beta_1) \mathbf{g} \quad (10)$$

$$\mathbf{g} = \beta_2 \mathbf{r} + (1 - \beta_2) \mathbf{g} \odot \mathbf{g} \quad (11)$$

$$\hat{\mathbf{s}} = \frac{\mathbf{s}}{1 - \rho_1^t} \quad (12)$$

$$\hat{\mathbf{r}} = \frac{\mathbf{r}}{1 - \rho_2^t} \quad (13)$$

$$\theta_t = \theta_{t-1} - \eta \frac{\hat{\mathbf{s}}}{\sqrt{\hat{\mathbf{r}} + \epsilon}} \quad (14)$$

Here ϵ is a small constant to avoid numerical errors, and $\odot$ denotes element-wise multiplication. We have used the default $\beta_1 = 0.9$ and $\beta_2 = 0.999$, as these are known to be well-performing for most models.

D. Loss functions

1. Cross Entropy

We use the PyTorch CrossEntropyLoss function for all models, as it is a well-known and standard function for multi-class classification. For a single review, each network will produce a vector of length 5, $\mathbf{x}_i = (x_{i,j})$, where the j -th entry is the raw unnormalized output of the neural network. If we let C be the true class label, the loss is then given as

$$\ell_i = -\log \left(\frac{\exp(x_{i,C})}{\sum_{i=0}^4 \exp(x_{i,j})} \right).$$

We use the mean reduction mode, so the loss score for an entire batch is given by

$$L = \frac{1}{N} \sum_{i=1}^N \ell_i \quad (15)$$

where N is the size of the batch.

E. Weight initialization

For the models to converge when training, we need some method of initializing the weights for the networks.

We can not simply let all initial weight be zero, as this would lead to all neurons learning the exact same features. We have used random initial weights to prevent this. For the RNN and LSTM models, we use the weights for each layer are randomly drawn from a range $[-\frac{1}{\sqrt{n}}, \frac{1}{\sqrt{n}}]$, where n is the layer size. The bag of words model is initialized with random weights from the uniform distribution over $[-0.5, 0.5]$.

F. Accuracy score

We use the accuracy score as a metric for our models, as it expresses the percentage of the test cases that the model correctly predicted. If we let $\mathbf{l} = (l_1, l_2, \dots, l_n)$ be the true labels for each review, and $\mathbf{p} = (p_1, p_2, \dots, p_n)$ be the predicted label, the accuracy is given by

$$\text{Accuracy} = \frac{\sum_{i=1}^n I(p_i = l_i)}{n}. \quad (16)$$

Here I is the indicator function, taking the value 1 if the given expression is true, and 0 otherwise.

G. Confusion Matrix

While the accuracy score can broadly tell us how our models perform, there can often be weaknesses that get hidden by the simplicity of the accuracy. The confusion matrix provides a better insight into where the models perform well, and which areas it can be improved. For a classification problem with K distinct classes, the confusion matrix is defined as a $K \times K$ matrix C , where each entry C_{ij} denotes the number of observations from class i predicted to be in class j . We can visualize this matrix using a heatmap, allowing for easier analysis of the results.

The diagonal C_{ii} of the matrix will now represent all correct classifications the model made, the true positives. All off-diagonal elements C_{ij} , $i \neq j$ represent the misclassifications of the model. For our purposes, we mainly want to reduce the severe misclassifications, which can be found at the upper right and lower left corners of the matrix (e.g when a 1 star review has been predicted as 5). The general distribution within the matrix also tells us about any potential biases that might not be apparent from the accuracy score alone.

H. Bias-Variance

When doing supervised machine learning, we wish to find a model that can accurately predict on data outside the training set, and thereby minimize the error we get when generalizing to more data. When doing regression with mean squared error, one can show that the error can be written as the sum of the bias, variance, and the

irreducible error.[6] While this does not hold in general for our problem of classification, it is still a good intuition for where the error of our models comes from.

The bias is a measure of the difference between our models predicted labels, and the true labels. If we have a model that is too simple, or is not sufficiently trained, it might not be able to pick up on patterns that it can use to correctly predict true labels. This would be a case of an underfitted model, which we could see if the model has bad performance on all data (training, validation, and testing sets).

The variance is the measure of how much the model's predictions would change if we slightly change the training set. A model with high variance has potentially become too specialized for the particular data it is trained on, including the random "noise" or particular tendencies that are present only in the training data. If we have a model that is too complex, or we train it for too many epochs, we might get that our networks will begin to "memorize" tendencies particular to the training data. This will lead to an overfitted model, where the performance on the training data is much better than on data in the validation or test set.

When balancing these two problems, we will often want to find a model that can both pick up on trends it finds in the training data, but still not become too specialized to the particulars of only the training set. When we increase the complexity of our models and train them for more epochs, we might see the error decreasing to a certain point, as the models are better equipped to pick up on trends. Beyond this however, we might see a rise in the error again, as the models become too specialized on the training set, and generalize poorly. Finding the optimal balance between the bias and variance becomes a difficult problem, as it might be hard to know at what point to stop training to prevent overfitting, but still train enough to get good results.

I. Recurrent Neural Networks

Recurrent Neural Networks (RNNs) are a class of neural networks specifically designed to process sequential data, such as time series, speech, or text by incorporating loops that allow information to persist across steps. Unlike traditional feedforward neural networks, which process inputs independently, RNNs maintain a hidden state that evolves over time, enabling them to capture temporal dependencies and context from previous elements in the sequence.

The core mechanism of an RNN involves processing input sequences one element at a time. For a sequence of inputs $x_1, x_2, \dots, x_T$, where each $x_t \in \mathbb{R}^d$ (e.g., a word embedding vector of dimension d), the hidden state $h_t \in \mathbb{R}^h$ (with hidden dimension H) at time step t is computed as:

$$h_t = f(W_{xh}x_t + W_{hh}h_{t-1} + b_h),$$

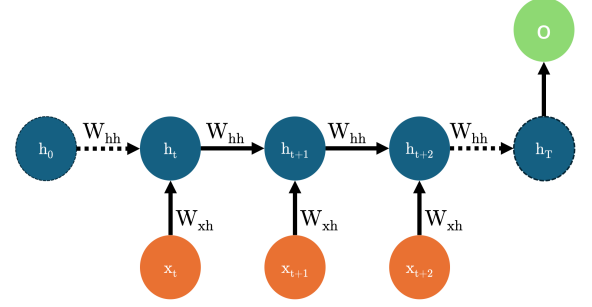


Figure 2. Recurrent structure of the hidden state in RNNs, through time. o is the final output of the recurrence.

where:

- f is a non-linear activation function, in our case the ReLU function.
- $W_{xh} \in \mathbb{R}^{H \times d}$ is the weight matrix for the connections from the input- to the hidden layer.
- $W_{hh} \in \mathbb{R}^{H \times h}$ is the weight matrix for the connections in the recurrent hidden layer connections.
- $b_h \in \mathbb{R}^H$ is the bias term.

Figure 2 illustrates the recurring process of the RNN through the time steps. the last hidden layer h_T is passed through a fully connected neural network to produce an output classifying the review into the classes 1 – 5.

J. Long Short-Term Memory

Standard RNNs often struggle when the input sequences get very long. As the network tries to learn dependencies that are far apart in the text, the gradients can become extremely small during backpropagation. This effectively causes the network to "forget" information from the beginning of a long review by the time it reaches the end.

Long Short-Term Memory (LSTM) networks are a variant of RNNs explicitly designed to solve this long term dependency problem. The key innovation in an LSTM is the cell state c_t . It creates a direct path for information to flow through the network unchanged, making it much easier for the model to retain context over long sequences.

The LSTM regulates the flow of information into and out of this cell state using gates. A standard LSTM unit uses three specific gates to control the cell state:

- f_t , the forget gate which decides what information to discard from the previous cell state.
- i_t , the input gate which decides which new information from the current step is relevant enough to store in the cell state.

- o_t , the output gate which decides what the next hidden state should be based on the current cell state.

For a given time step t with input x_t and previous hidden state h_{t-1} , the gates and states are updated as follows:

First, the gates are computed using the sigmoid function σ , which determines how much of the information to pass through the gate.

$$f_t = \sigma(W_{xf}x_t + W_{hf}h_{t-1} + b_f) \quad (17)$$

$$i_t = \sigma(W_{xi}x_t + W_{hi}h_{t-1} + b_i) \quad (18)$$

$$o_t = \sigma(W_{xo}x_t + W_{ho}h_{t-1} + b_o) \quad (19)$$

Next, we calculate a candidate for the new cell memory using the tanh activation function:

$$g_t = \tanh(W_{xc}x_t + W_{hc}h_{t-1} + b_c) \quad (20)$$

We then update the cell state by forgetting the old information and adding the new candidate memory:

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t \quad (21)$$

Finally, the new hidden state is a filtered version of the cell state:

$$h_t = o_t \odot \tanh(c_t) \quad (22)$$

Here, the operator $\odot$ denotes element-wise multiplication. By using this gating system, the LSTM can choose to remember a specific sentiment word (like "terrible" or "awesome") seen at the start of a review use it to correctly classify the review at the very end of the sequence. Like with normal RNNs, the final hidden layer is connected up to a fully connected neural network which produces a prediction of the label of the review.

K. Feed-Forward Neural Networks and Bag of Words

The Bag of Words (BoW) model is one of the simplest and most common techniques for processing text in Natural Language Processing. The model processes a sentence like a bag of its individual words, disregarding the order they come in the sentence. The only information that is stored are the words in the sentence, making sentence structure entirely irrelevant.

We first construct a vocabulary of all unique words found in the entire training dataset. From this, each review can be converted into a vector with length of the number of unique words in the vocabulary. The value of an entry in the vector is the count of how many times that specific word appears in the current review.

This vector is then fed into a simple feed-forward neural network, producing an output vector with a label prediction. The neural networks consists of layers of neurons, connected by weight matrices and activation functions between each layer. We have previously shown that these networks are extremely adaptive and suitable for most tasks which require some learned behavior, like regression or classification.[6] Due to the simplicity of this model, it is extremely efficient and easy to implement and train. The problem with this is that by removing all order of the sentence, we might lose important context, as the meaning of individual words are often influenced by their preceding words.

L. Implementation

1. Data handling

All reviews are preprocessed as described previously, reducing the size of the alphabet and tokenizing into individual words. The data is already split into a training and test split, however we again split the training dataset into a set we use for model training (containing 80% of the original training set) and a validation set (containing 20%). The validation subset can be used for more advanced training methods like k -fold cross validation, however we have only used it for monitoring the performance and potential overfitting of the networks during training. The reviews are put in PyTorch datapipe, and collated into batches of size 2048, to closely match the setup used by Zhang et al. [1], making comparisons of the results easier. We shuffle the training data at the beginning of each epoch, ensuring that we are using stochastic methods.

Due to the size of the dataset, preprocessing all reviews takes a significant amount of time for the whole dataset. We have implemented methods for local caching of the preprocessed dataset, and a method for generating smaller subsets of the entire dataset for testing purposes.

2. Models

For the RNN and LSTM models, we have used the same network architecture as the example given in Raschka et al.[4] Due to the significant computational expense and time of training the models, we have not been able to fully explore different model architectures, however the current setup provides adequate results for our comparison purposes. The setup for both models has an embedding layer of dimension 20, an RNN / LSTM layer with an output dimension of 128 nodes, a fully connected layer with 128 nodes, and finally the output layer of 5 nodes. The only non-linear activation function is between the fully connected and output layer, with a ReLU activation.

The bag of words model is intentionally kept extremely simple, having no hidden layers between the input and output layers. This is so we can compare the more complex models with a standard well-known method in sentiment analysis. The model is also trained using a much simpler optimizer. This is due to the fact that the simple model provides a much smoother optimization landscape, as compared to the more complex RNN models. The model consists of a layer which bags the words, and a fully connected layer connecting to the 5 output nodes.

3. Hardware

The models used in this project are trained on the University of Oslo’s ML Nodes [7]. The server used for training contains 4 Nvidia 2080Ti GPUs, each with 11 GB of RAM. This setup provides a significant speedup from running locally on CPUs (around a 50 times speedup), which makes training with the large dataset feasible.

M. Use of AI tools

We have used AI tools to aid in the writing of this project. None of the text is directly generated from the LLM, but it has been reviewed and improved upon using suggestions from the AI. The AI has also been used for some code generation, code and $\text{\LaTeX}$ debugging. We have tried to not use an excess amount of AI, as the climate impact of large language models is not insignificant. For our AI, we have used the locally hosted GPT UiO, which uses models from OpenAI. We have used this model as it is the most privacy-focused option available. All conversations with the AI can be found in the project repository

III. RESULTS AND DISCUSSION

A. Training and the Bias-Variance tradeoff

The plot of the cross-entropy loss during training, as shown in Figure 3, shows large differences in the performance of the models throughout training over many epochs. We get a good insight into the bias-variance tradeoff for each of the three models, and how it affects how we should train them for optimal performance.

The Bag of Words model exhibits the most stable training performance. Both the training and validation loss decrease steadily and converge relatively quickly. The close proximity of the training and validation curves suggests that the BoW model achieves a good balance between bias and variance. It is complex enough to learn the dominant sentiment keywords but simple enough to avoid memorizing specific “noise” and patterns in the training data.

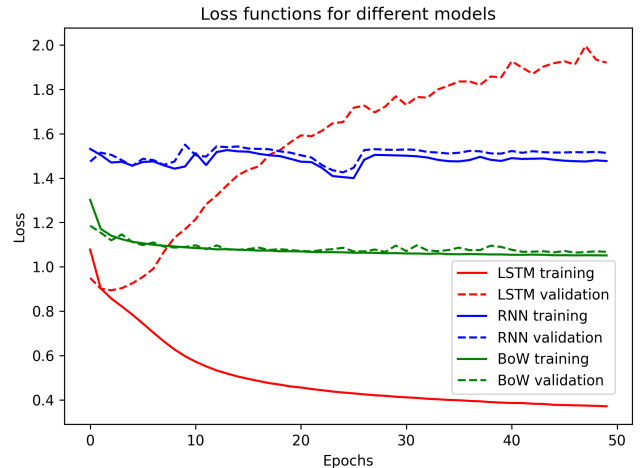


Figure 3. Loss for training and validation data on different models

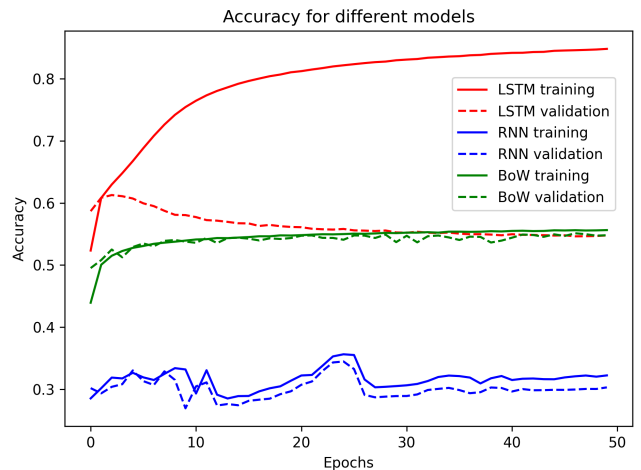


Figure 4. Validation set accuracy score for models throughout training.

The Recurrent Neural Network displays clear signs of underfitting. As observed in Figure 3, the loss for the RNN remains high (around 1.5) throughout the entire training process, with the accuracy curve shown in Figure 4 showing a similar inability to improve beyond an initial accuracy of around 30%. This failure can be attributed to the vanishing gradient problem, where some neurons are unable to learn due to their gradients becoming too small. Because the simple RNN struggles to propagate errors back through long sequences, it struggles with picking up patterns that might be important from the beginning of reviews. We thereby get a model that is functionally unable to classify text well, and no amount of training can help this problem. This also means that we need not worry about overfitting when training this model for any reasonable amount of epochs, as it is simply unable to be overfitted due to the problems described.

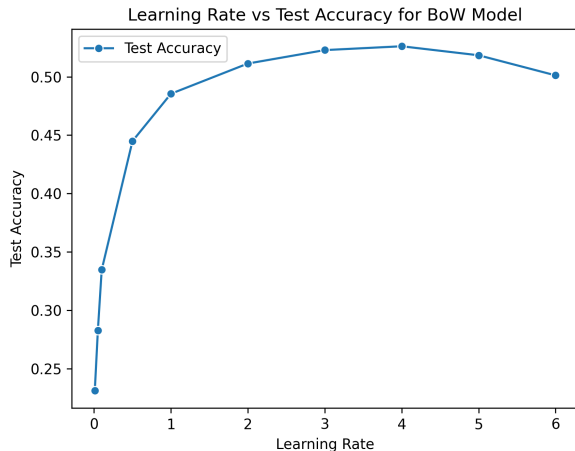


Figure 5. Learning Rates and Test Accuracy for BoW model using SGD, 3 epochs.

The Long Short-Term Memory network suffers from the opposite problem, with it showing clear signs of overfitting even after only a few epochs. While it achieves the lowest training loss and highest accuracy of all the models, we see from Figure 3 that the loss for the training and validation sets begin to diverge after about 3 epochs. The training loss continues to steadily decrease with more training, the validation loss also shows a steady increase over the same time. This is a clear illustration of the overfitting we described in section II H. The gating mechanism of the LSTM model makes it so that it is able to pick up on important words early in reviews, thereby fixing the problems we see with the plain RNN. This does however also allow for the model to learn trends and patterns only present in the training data more easily, thus enabling the possibility of overfitting. While we see from Figure 4 that the LSTM model is by far the best performing in terms of accuracy, we need to ensure that we do not train it to the point of overfitting, as to not degrade the performance on the test set.

Moving forward, we will train our models for 3 epochs, as beyond this point, there is no observable benefit in terms of accuracy to be gained from further training.

We have also looked at what effect the learning rate has on the model performance. As Adam is an adaptive learning rate method, the performance of the final trained model will be much less dependent on the initial learning rate. This is however not the case for our bag of words model, which is trained using a fixed learning rate. Figure 5 shows the performance of the BoW model for different learning rates. We see that for small learning rates, there is a significant decrease in the model performance, which is likely due to the model being underfitted due to the low learning rate. The same can be observed for high learning rates $\eta > 4$, where we also see a drop in performance. This might be due to the optimizer having to jump too far in the optimization landscape, poten-

	Test Accuracy	Training time	Epochs	Parameters
RNN	0.306045	1121s	3	15392597
LSTM	0.613225	1149s	3	15450197
BoW	0.525577	1047s	3	49140293

Table I. Test accuracy, training time in seconds, epochs trained, and number of trainable parameters for each model

tially hindering good convergence. Our learning rate of $\eta = 4.0$ which we used when training over many epochs seems to be somewhat optimal, at least over the values tested.

B. Model performance

Table I summarizes the final performance metrics for the three trained models evaluated on the test set. As before, we see a clear difference in which models perform best. The Long Short-Term Memory network achieves the highest accuracy of 61.2%, significantly outperforming the plain Recurrent Neural Network, which is about half as good at 30.1% accuracy. Interestingly, the simple Bag of Words (BoW) model still performs relatively well, achieving an accuracy of 54.8%. This suggests that for this dataset, the presence of certain words can be effectively used to determine a review score quite well. The ordering which the LSTM model preserves only provides a small benefit in performance, coming at a cost of about 100 seconds of training time. This is due to the backpropagation through the time layers of the LSTM model, something which takes much less time with the one-layer BoW model. We also see that even though the BoW model is conceptually the simplest, it has by far the most trainable parameters. This is due to the fact that the size of the vocabulary is so large, making the size of the fully connected FFNN large. It should be possible to reduce the size of the vocabulary using methods like spelling corrections and augmenting the data using a thesaurus, as described in [1]. Lastly, the RNN model is by far the worst model, having both the drawback of long training times due to the expensive backpropagation, as well as not being able to learn effectively, only being slightly better than guessing randomly.

Looking closer at Figure 6 reveals that the RNN model has a significant bias towards predicting an extremal value of either 1 or 5. This suggests that the RNN might be better suited for a binary class problem, like if we were to classify the reviews as either good or bad. Due to the nature of how the model is predicting values, we also see a relatively high rate of large misses, particularly 1 star reviews being classed as 5 stars. This shows a clear inability for the model to pick up on certain trends which are characteristic of bad reviews, in that reviews on the opposite sides of the scale should have enough discerning characteristics to be easily separated. There are potential methods to fix some of the issues we encounter with vanishing gradients, particularly using gradient clipping.

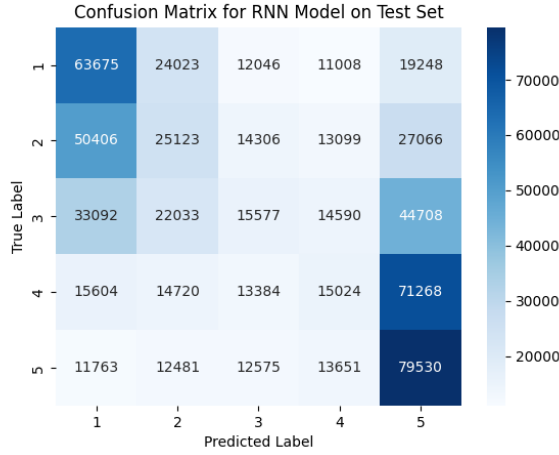


Figure 6. Confusion matrix for Recurrent Neural Network model, trained for 3 epochs

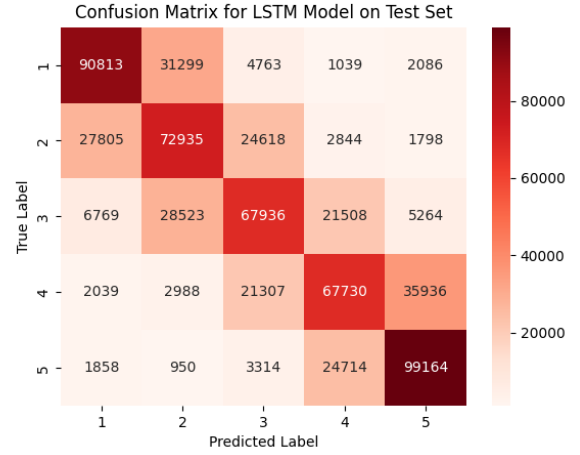


Figure 8. Confusion matrix for LSTM model, trained for 3 epochs

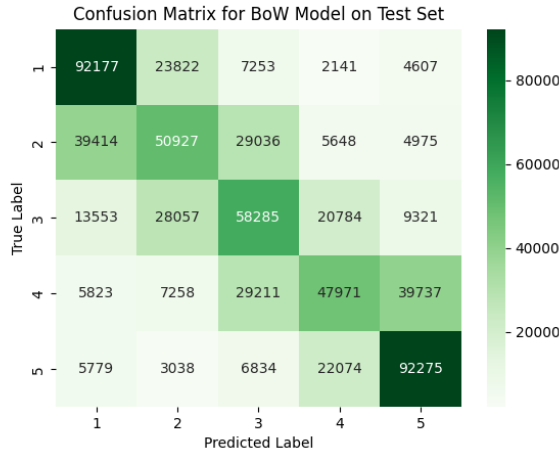


Figure 7. Confusion matrix for Bag of Words model, trained for 3 epochs

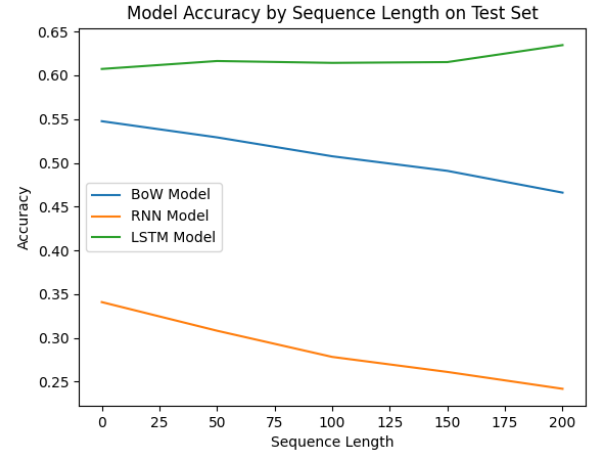


Figure 9. Model accuracy for test set by sequence length

For the LSTM and BoW models, we can look at figures 8 and 7 respectively. Here we see a much more even distribution of guesses across the diagonal of the matrices. Both models are still somewhat biased towards extremal values (1 and 5), however they are also much more likely to guess a different label, with the distribution of the predictions being more even over the classes. The occurrences of large misses are also greatly reduced, with most reviews being predicted within a margin of ± 1 stars. We also see that the extra 9% gain in accuracy for the LSTM model over the BoW model comes from the increased accuracy on more neutral reviews, with star ratings in the 2 – 4 range. This suggests that for reviews that are not rated 1 or 5, the additional context provided by the ordering of words provides a better framework for the model to learn effectively.

It is also interesting to note that both matrices are

somewhat symmetric around the diagonal. There are many reasons as to why this might be, however the most obvious is the fact that for neutral reviews, there are fewer words that strongly indicate a rating. This makes large misclassifications less common, but the closer we get to the true label of a review, the more the details of word choice and structure matter. If we consider reviews that are predicted to within 1 star of the review’s actual rating, the LSTM model classifies 94.5% of reviews into what we might deem as an acceptable range.

Figure 9 illustrates the problems with the RNN and BoW models. There is a clear trend towards longer reviews degrading both models performance, whereas the LSTM model has no indication of degradation for long sequences. For RNNs, this is likely due to the fact that tokens early in the sequence are almost forgotten towards the end of long reviews. If there are many words indicative of a particular class early in a review, the RNN will

not be able to pick up on these, as compared to the LSTM model, which will remember certain high impact words with its gating system. As to why the BoW model performs worse with longer sequences, the relative frequency of important words might be affected with longer reviews, potentially skewing the predictions.

C. Comparisons

For a point of comparison, we can look to Zhang et al.[1], who have trained a variety of models on the same dataset. Their best performing model was a small Convolutional Neural Network, using a thesaurus to merge similar words, with an accuracy score of 59.57%. Only our LSTM model can slightly outperform this, indicating that recurrent neural networks might be better for sentiment analysis. For a more direct comparison, their LSTM had an accuracy of 59.43%. The performance boost for our model is likely due to small choices in optimizer, alphabet, and learning rates.

Our bag of words model is however worse than the one presented in [1], with their model 54.64% accuracy rate. This may be due to the fact that they restrict their bag of words to the 50 000 most common words in the vocabulary, whereas we have no such restriction. This makes the network smaller and less complex, potentially helping the accuracy. There may also be small differences in the training process that lead to the increased performance. More complex models, like one produced by Yin et al. [8], still outperform our models by a relatively large margin. They present a character-level attention convolutional neural network (ACNN), which gets an accuracy of 65.63%, suggesting LSTMs might not be optimal for this task..

IV. CONCLUSION

In this project, we have explored the challenges of multiclass sentiment analysis by implementing and compar-

ing three distinct architectures: a Bag of Words model, a Recurrent Neural Network, and a Long Short-Term Memory network. By training these models on a dataset of 3 million Amazon reviews, we were able to determine the suitability of each of the architectures for the class of text sentiment analysis, as well as details relating to the training of these models.

Our results conclusively demonstrate the limitations of simple Recurrent Neural Networks for long-sequence tasks. With an accuracy of only 30%, the standard RNN failed to capture meaningful dependencies in the text, performing only marginally better than a random guess. Further exploration is necessary to find methods to remedy this, with methods like gradient clipping. This failure highlights the severity of the vanishing gradient problem in practice, as the network was unable to retain context from the beginning of the reviews. In contrast, the LSTM network successfully mitigated this issue through its gating mechanisms, achieving the highest test accuracy of 61.2%. However, this increased capacity could come at the cost of significant overfitting, with validation performance degrading while training performance continued to improve.

Interestingly, the Bag of Words model proved to be a robust and efficient baseline, achieving 54.8% accuracy with a fraction of the computational cost required for the LSTM. This suggests that while sequential modeling is necessary to capture the full nuance of human language, a significant portion of sentiment can still be derived from the presence of specific words.

For future work, the clear overfitting observed in the LSTM suggests that stronger regularization techniques, such as dropout or weight decay, are necessary to fully exploit the model's capacity. Additionally, implementing further preprocessing of the data through a thesaurus might help boost performance further. Finally, from other research we know that Convolutional Neural Networks like ACNNs might perform better than any of the sequential models we have studied.

-
- [1] X. Zhang, J. Zhao, and Y. LeCun, Character-level convolutional networks for text classification (2016), arXiv:1509.01626 [cs.LG].
 - [2] J. Leskovec and A. Krevl, SNAP Datasets: Stanford large network dataset collection, <http://snap.stanford.edu/data> (2014).
 - [3] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, Pytorch: An imperative style, high-performance deep learning library (2019), arXiv:1912.01703 [cs.LG].
 - [4] S. Raschka, Y. H. Liu, and V. Mirjalili, *Machine Learning with PyTorch and Scikit-Learn* (Packt Publishing, Birmingham, UK, 2022).
 - [5] D. P. Kingma and J. Ba, Adam: A method for stochastic optimization (2017), arXiv:1412.6980 [cs.LG].
 - [6] S. Oppen and O. M. Bråthen, Linear regression and resampling techniques on the runge function, (2025).
 - [7] University Centre for Information Technology, University Of Oslo, Machine learning infrastructure (ml nodes) (2025).
 - [8] F. Yin, Z. Yao, and J. Liu, Character-level attention convolutional neural networks for short-text classification, in *Human Centered Computing*, edited by D. Milošević, Y. Tang, and Q. Zu (Springer International Publishing, Cham, 2019) pp. 560–567.